After Launch, the Meter Still Runs

Where This Booklet Fits:
This is booklet 5 of 5 in the Don’t Go Broke series.

Disclaimer: This booklet provides general educational information and is not formal financial, legal, or professional engineering advice.

Table of Contents

After Launch, the Meter Still Runs

Field Note Opening: The Finish Line Is the Starting Line

You built it. You reviewed the draft. You fixed the bugs. You pushed the final button, and now your AI-generated app is live on the internet.

In that moment, there is a profound sense of relief. You survived the blank page, the frustrating prompts, the hallucinated architecture, and the confusing errors. It is natural to feel like you have crossed the finish line.

But building a software product is a project with an end date; operating a software product is a commitment with no scheduled conclusion. Launch is not the end of the work. It is the exact moment when the operational reality—and the recurring costs—begin.

When you were building, you were spending time. Now that the app is live, you are spending money.

This booklet is about what happens next. It is about how to survive the first week, how to find the hidden meters that are quietly spinning up costs, and how to operate your live app without letting it bankrupt you.

---

1. The Invisible Meters

You are no longer paying for a one-time product. You are renting infrastructure by the millisecond. Every time a user clicks a button, a tiny invisible meter ticks up.

There are four main meters that run continuously:

1. **Compute:** The server power required to run your code.
2. **Storage and Logs:** The database space required to save your users' information, and the space required to save system error records.
3. **Third-Party APIs:** External services (like sending emails, processing payments, or fetching weather data) that charge you per event.
4. **AI Tokens:** The cost of asking the AI model to process text or generate an answer for your users.

AI makes building fast and cheap, but it does not make running the app free. If your app becomes popular overnight, those invisible meters will spin faster than you can blink.

The After-Launch Cost Map Worksheet

Before you can control your costs, you must map them. Use this worksheet to document exactly where your app is spending money.

- ☐ **Hosting Provider:** Where does the code live? (e.g., Vercel, Heroku, AWS)
  - *Billing Tier:* _____
  - *Hard Cap Set At:* $_____
- ☐ **Database Provider:** Where is the user data stored? (e.g., Supabase, MongoDB, Firebase)
  - *Billing Tier:* _____
  - *Hard Cap Set At:* $_____
- ☐ **AI Provider:** Who provides the intelligence? (e.g., OpenAI, Anthropic, Gemini)
  - *Billing Tier:* _____
  - *Hard Cap Set At:* $_____
- ☐ **Email/SMS Provider:** Who sends the notifications? (e.g., Resend, Twilio)
  - *Billing Tier:* _____
  - *Hard Cap Set At:* $_____

"Where Can This Keep Spending Money?" Prompt

If you used an AI agent to build the app, it likely integrated services you aren't fully aware of. Copy and paste this prompt to interrogate the AI about your live architecture:

> "My application is now live. I need to map all recurring operational costs.
>
> 1. List every third-party service, API, database, and hosting platform this codebase connects to.
> 2. For each service, explain exactly what user action triggers a cost. (e.g., 'Every time a user creates a post, it burns AI tokens and uses database storage.')
> 3. Identify any features that could easily be abused by a user to intentionally or accidentally spike my billing."

---

2. Background Jobs and Silent Loops

The most dangerous cost in a modern application is the one that happens while nobody is using it.

When you ask an AI to build a complex feature, it will often write a "background job"—a process that runs automatically on a schedule, behind the scenes.

Quiet Cost Leak: A Worked Example

A founder asked an AI to build a feature that pulls new articles from an RSS feed and summarizes them. The AI wrote a background job (a "cron job") to check the feed every 60 seconds.

The founder launched the app on a Friday and went away for the weekend. The app had exactly zero users. But every 60 seconds, the background job woke up, fetched data, called the AI API to attempt a summary, failed due to a missing article, wrote the error to a log file, and went back to sleep.

What Looked Safe: The app was live, the UI was clean, and there were no users clicking buttons. The dashboard looked calm.
Where the Cost Came From: A silent, scheduled retry loop that triggered a third-party AI call and generated a database write

every 60 seconds. **The Result:** By Monday, the app had run 4,320 times. It burned through the founder's AI API credits and filled up the database's storage limit with 4,320 identical error logs. The bill was over $200, all for an app with no users.

The Quiet Cost Leak Diagnostic

You must audit your app for silent loops immediately after launch.

- ☐ **Scheduled Tasks:** Are there any cron jobs running on a schedule? (Every minute, every hour, every day?)
- ☐ **Retry Logic:** If a task fails (like an email failing to send), does it try again forever?
- ☐ **Webhooks:** Is your app waiting for external events (like a payment processor confirming a transaction) to trigger massive database updates?

"Find Hidden Recurring Work" Prompt

Use this prompt to force the AI to reveal its hidden background processes:

> "Audit this codebase for background processes.

> 1. List every cron job, scheduled task, or background worker that runs automatically without a user clicking a button.
> 2. List every function that contains automatic 'retry' logic if it fails.
> 3. List every webhook or listener waiting for third-party API events.
> 4. For each item, explain what it costs to run and what happens if it gets stuck in an infinite loop."

---

3. Logs, Storage, and Database Review

Compute and AI tokens get all the attention, but boring infrastructure can bankrupt you just as quickly.

The Logging Trap Every time an error happens, modern apps write a "log." If your app has a bug that triggers an error every time a user scrolls, your app might generate 10,000 logs a day. Log storage is not free. If you do not monitor your logs, your database bill will skyrocket just from storing records of your own app's failures.

The File Upload Trap If your app allows users to upload profile pictures or PDFs, you are paying for storage. What happens if a malicious user uploads a 5-gigabyte movie file? Did the AI put a size limit on the upload form? If not, you are paying for their movie storage.

The Email Trap Every password reset email, welcome email, and notification costs a fraction of a cent. If a bot discovers your signup form and creates 10,000 fake accounts, you will pay for 10,000 welcome emails.

Logs / Storage / Database Review Sheet

Use this beginner-safe workflow to review your storage infrastructure:

- [ ] **Check Log Growth:** Are logs growing quickly? Are old error logs automatically deleted after 7 days to save space?
- [ ] **Check Upload Limits:** Are file uploads strictly capped at a safe size (e.g., 5MB), and are files deleted when a user deletes their account?
- [ ] **Check Failed Records:** Are failed records (like unsent emails) piling up in a queue table indefinitely?
- [ ] **Check Test Data:** Was test data left in production, taking up valuable database space?
- [ ] **Check Generated Content:** Are old AI-generated text results being saved forever even if the user abandons them?

"What Can Grow While I Sleep?" Prompt

> "Review the database schema and file storage logic in this codebase.

> 1. List every table or bucket where data is added automatically (e.g., error logs, system events).
> 2. List every place where user-uploaded files are stored.
> 3. Tell me exactly when this data is deleted. If there is no deletion logic, write a script to automatically delete logs older than 7 days and files belonging to deleted users."

---

4. AI Credit Usage After Launch

If you built an app that relies on an AI API (like OpenAI or Anthropic), you face a unique operational risk.

Traditional APIs are cheap and predictable. Large Language Model (LLM) APIs are expensive and highly variable. The cost depends entirely on how much text the user types and how much text the AI generates.

Scenario: The Uncapped Feature

A builder launched an AI writing assistant. The AI charged per token (word). The builder assumed typical users would write a few paragraphs a day.

Instead, a small group of three power users pasted entire 500-page novels into the tool, asking the AI to rewrite them. The builder had not set a character limit on the text input box. Those three users burned through the builder's entire monthly budget in a single afternoon. A "successful" launch with highly engaged users resulted in a devastating bill because the feature was left uncapped.

"Find Uncapped Cost Paths" Prompt

- [ ] **Input Limits:** Is there a strict character limit on every text box that talks to the AI?
- [ ] **Output Limits:** Is the AI's response capped at a specific maximum length (e.g., `max_tokens: 500`)?
- [ ] **Rate Limiting:** Is a user prevented from clicking the "Generate" button 50 times in one minute?

> "Audit this codebase for uncapped AI and API usage.
>
> 1. List every text input that is sent to an AI API. Does it have a strict character limit?
> 2. List every API call that is triggered by a button click. Does it have rate limiting (e.g., max 5 clicks per minute)?
> 3. If any inputs or buttons are uncapped, generate the code to restrict them immediately."

---

5. Usage Spikes and Surprise Bills

The most dangerous assumption a first-time builder can make is, "I'll just check the billing dashboard every few days."

Do not trust your future self to remember. An unexpected traffic spike, a bot caught in a loop, or a poorly optimized database query can rack up hundreds of dollars in costs while you are sleeping.

You must set traps.

The Budget Alert Setup Checklist

Before the first user arrives, go into your hosting and AI provider dashboards and set up strict budgets. Complete this checklist across every provider:

- ☐ **Hosting Budget Alert:** Alert set at $**. Hard limit set at $**.
- ☐ **Database/Storage Alert:** Alert set at $**. Hard limit set at $**.
- ☐ **AI/API Credit Limit:** Alert set at $**. Hard limit set at $**.
- ☐ **Email/Send Limit:** Alert set at _____ emails/day.
- ☐ **Logging Retention Limit:** Logs automatically delete after _____ days.
- ☐ **Payment Provider Alert:** If a payment processor like Stripe fails repeatedly, does it alert me?
- ☐ **Weekly Manual Review Reminder:** Calendar event set to check billing dashboards manually every Friday.
- ☐ **Emergency Contact:** Do I know which credit card is attached to these accounts if I need to cancel it?

The Usage Spike Triage Sheet

When you get an alert that a limit has been reached, do not panic and simply raise the credit card limit. Pause before celebrating a traffic spike. Execute this triage plan:

- [ ] **Identify the Source:** Which meter spiked? (Compute, Storage, Emails, or AI Tokens?)
- [ ] **Separate Good from Expensive Usage:** Are real users paying you, or are free users burning expensive API credits?
- [ ] **Check for Bots:** Is it one IP address making 10,000 requests?
- [ ] **Check for Loops:** Did a background job get stuck retrying a failed task?
- [ ] **Temporarily Cap Features:** Turn off the most expensive feature immediately to stop the bleeding while you investigate.
- [ ] **Document the Incident:** Write down exactly what happened so you don't build a new feature with the same vulnerability.

---

6. The First Week Operating Rhythm

The first day after launch is not about celebrating; it is about surviving. Your app is hitting the real world for the first time. You must actively monitor its vital signs over the first seven days.

The First Week Operating Log

Walk through this rhythm every day for the first week:

- **Day 0: The Launch.** Confirm the app is live. Verify that no obvious runaway processes started the moment you deployed. Ensure all budget alerts are active. Do not add any new features today.
- **Day 1: The Initial Check.** Check the hosting dashboard, the database size, the AI/API credit usage, the email send logs, and the system error logs. Are there any massive walls of red error text? Did you actually receive the test alert you set up?
- **Day 2–3: The Hidden Fault Lines.** Look deeper. Are there repeated errors? Are background jobs failing and retrying? Are there any unexpected spikes in compute or storage from a small group of users?
- **Day 4–5: The Reality Check.** Compare real usage against your expected usage. Are users clicking buttons 10 times more often than you predicted? Are they uploading larger files? Calculate your daily run rate based on the last four days.
- **Day 6–7: The Decision Gate.** Look at the full week of data. Decide whether you can afford to keep operating at this pace. Decide whether you need to pause expensive features, add stricter limits, or continue cautiously.

---

7. The Safe Feature Addition Gate

By Day 7, the initial launch excitement will fade, and users will start asking for new things. The reality of maintenance begins here.

Builders often make the mistake of adding complex new features before understanding the operating cost of the existing ones. Do not let momentum blind you.

The Safe Feature Addition Gate

No new feature should be added immediately after launch until you can answer every question on this list:

- [ ] What does the current app cost per day to run?
- [ ] What user action triggers the highest cost?
- [ ] What part of the app's infrastructure is currently the most expensive?
- [ ] What hard limits are currently protecting my credit card?
- [ ] What happens to my bill if usage doubles tomorrow?
- [ ] What happens if the background jobs associated with this new feature fail and retry forever?
- [ ] What happens if the AI/API calls in this new feature get abused by a bot?

If you cannot answer these questions, you are not ready to build a new feature. You must master operating the current app first.

---

8. Pause, Scale Down, Keep Going, or Shut Off

There is a taboo around killing a project. Many builders will spend fifty dollars a month keeping an abandoned app alive just because they feel guilty about shutting it down.

If the recurring costs outweigh the value the app provides, you have to make a hard decision. Do not let zombie apps slowly drain your bank account.

The Pause / Scale Down / Shut Off Playbook

Use this decision guide to control an app that is no longer behaving exactly as you want:

1. PAUSE (The Emergency Brake) *When to use it:* A bug is burning through your API credits, or a bot is spamming your database. *Action:* Pause a feature, but keep the app live. You can temporarily disable the "Generate AI" button, or temporarily revoke the third-party API keys to stop the bleeding. Fix the code locally, then turn it back on.

2. SCALE DOWN (The Baseline Retreat) *When to use it:* The app is stable, but nobody is using the expensive features, or

you can no longer afford the current tier. *Action:* Turn off AI generation entirely but keep the previously saved results visible. Reduce logging retention from 30 days to 7 days. Disable background jobs temporarily. Cap file uploads. Downgrade your database from the $20/month tier to the $5/month tier. Lower the baseline cost.

3. KEEP GOING (The Green Light) *When to use it:* The app is generating enough value (or revenue) to easily cover its run rate. The logs are clean. The background jobs are sleeping appropriately. Budget alerts are active. *Action:* Keep going only after alerts and limits are in place. Proceed to the next phase of the product lifecycle. You have earned the right to start building new features.

4. SHUT OFF (The Graceful Shutdown) *When to use it:* The app costs more to run than it earns. You have lost interest in maintaining it. The codebase has rotted. *Action:* Shut down a demo app cleanly. Give your users 30 days notice. Let them export their data. Then cleanly delete the database, cancel the hosting, and revoke the API keys.

---

9. The Safe Operation Checklist

Operating an app means managing a living system. Use this final protocol to ensure your system is under control before you walk away from the keyboard.

- ☐ **The Meters Are Visible:** I know exactly where this app spends money.
- ☐ **The Traps Are Set:** Hard billing limits are actively protecting my credit card.
- ☐ **The Background is Quiet:** I have audited every cron job, retry loop, and webhook.
- ☐ **The Edges Are Capped:** All AI inputs, file uploads, and API calls have strict maximum limits.
- ☐ **The Shutdown Plan Exists:** I know exactly how to pull the plug if things escalate.

Field Note Closing

Building an app with AI is an incredible achievement. Launching it takes courage. But operating it requires maturity.

By setting hard billing limits, hunting down silent background jobs, and anticipating the true cost of users, you transition from someone who just got lucky with an AI builder to someone who knows how to run a software business.

You survived the build. Now you are equipped to survive the operation.

The meter is running. Keep your eyes open, watch the limits, and operate with confidence.

The Next Step

Congratulations, you have completed the series!